EventHub
CONNECT. PARTICIPATE. EXPERIENCE.

PROJETO ACADÊMICO | UECE

Relatório Final do Projeto EventHub

Plataforma de gerenciamento de eventos acadêmicos e profissionais

---

ARQUITETURA
Microsserviços com comunicação síncrona e assíncrona

NOTA DE ESCOPO

Este documento descreve oito serviços de domínio implementados e um provedor de pagamento simulado separado. A instituição é a conta organizadora de seus eventos, e os certificados são por atividade. Os testes integrados utilizam ambiente próprio de QA. Resultados anteriores são identificados como evidências históricas, sem atribuir nova execução ou publicação remota não comprovada.

Universidade Estadual do Ceará - UECE
Relatório técnico | 11 de novembro de 2026

Sumário

1 Visão geral do projeto 2

2 Problema que o projeto resolve 2

3 Objetivos 3

4 Público-alvo 3

5 Requisitos funcionais 4

6 Requisitos não funcionais 4

7 Histórico de decisões e evolução 5

8 Tecnologias utilizadas 5

9 Estrutura de diretórios 6

10 Arquitetura do sistema 7

11 Descrição detalhada dos módulos 9

12 Funcionamento das funcionalidades 11

13 Regras de negócio 12

14 Modelagem de dados 14

15 Fluxos de execução 17

16 Integrações externas 19

17 Configuração e execução 20

18 Testes e evidências de validação 21

19 Estado atual do desenvolvimento 23

20 Limitações e melhorias futuras 23

21 Comparação entre planejamento e estado atual 25

22 Encerramento e próximos passos recomendados 25

Referências técnicas da atualização 26

1. Visão geral do projeto

O EventHub é uma plataforma web para organizar eventos acadêmicos e profissionais, integrando participantes e instituições. A instituição é a conta organizadora: cadastra seus dados, cria e gerencia seus eventos, programa atividades, acompanha inscrições, registra presença e consulta indicadores.

A conta participante consulta eventos, inscreve-se em atividades, acompanha reservas e pagamentos simulados e obtém certificados. A arquitetura separa responsabilidades, propriedade dos dados e regras de domínio, utilizando Node.js e Python.

Grupo	Componentes atuais
Oito serviços de domínio	Identity, Events, Registrations, Payments, Notifications, Attendance, Certificates e Reporting.
Provedor simulado	Payment Provider, separado de Payments e com banco próprio; sem cobrança real.
Entrada e infraestrutura	Frontend React, Gateway, Nginx, PostgreSQL e RabbitMQ.

Attendance, Certificates e Reporting possuem implementação, contêineres e endpoints operacionais; não são apenas modelados. A composição principal possui nove bancos lógicos e 14 contêineres.

O texto descreve o workspace inspecionado em 11/11/2026, cuja referência Git é `ab70979`, com alterações locais preexistentes. Esse identificador não representa, isoladamente, todas as revisões dos registros históricos.

Evidência: `compose.yaml`; `docs/architecture/business-capabilities.md`; `infra/db/01-init.sh`.

2. Problema que o projeto resolve

O gerenciamento de um evento reúne identificação, publicação de atividades, controle de vagas, pagamentos, comunicação e comprovação de participação. Concorrência e falhas distribuídas exigem regras explícitas: duas pessoas podem disputar a última vaga, um pagamento chegar após a expiração ou uma notificação falhar depois da confirmação.

O EventHub coordena esses domínios por APIs e eventos, mantendo transações locais e recuperação posterior, sem compartilhar tabelas entre serviços.

- Localizar eventos e consultar atividades.
- Inscrever-se com estados e disponibilidade definidos pelo servidor.
- Permitir à instituição administrar seus próprios eventos.
- Acompanhar reservas e pagamentos simulados.
- Registrar presença, emitir certificados por atividade e consultar indicadores.

A solução demonstra separação de responsabilidades e também seus custos: consistência eventual, autorização entre serviços e necessidade de observabilidade.

3. Objetivos

O objetivo principal é disponibilizar um protótipo web integrado de eventos com microsserviços e pelo menos duas tecnologias de backend.

1. Implementar autenticação e autorização com perfis exclusivos de participante e conta institucional.
2. Permitir criação, edição de rascunhos, publicação e cancelamento dos eventos da instituição.
3. Organizar atividades com palestrantes, datas, capacidade, preços e espera configurável.
4. Evitar ocupação excedente e inscrições duplicadas.
5. Diferenciar confirmação gratuita e reserva paga temporária.
6. Confirmar pagamento simulado apenas quando a reserva permitir.
7. Liberar vagas após cancelamento, recusa e expiração, promovendo a espera quando aplicável.
8. Processar notificações sem bloquear a confirmação.
9. Registrar check-in e emitir certificados após a atividade, com presença válida.
10. Disponibilizar indicadores e exportações com valores financeiros simulados.
11. Manter interface baseada no Figma e execução com Docker Compose.
12. Separar o ambiente da aplicação do ambiente de testes integrados.
13. Documentar contratos, estados, decisões e limites das evidências.

A exposição pela internet pode ser configurada sobre o Nginx. Uma URL pública ou implantação remota não é inferida apenas da arquitetura.

4. Público-alvo

Público	Utilização
Participante	Consultar eventos, realizar inscrições, acompanhar reservas e pagamentos, manter perfil e acessar certificados próprios.
Instituição organizadora	Cadastrar a instituição e criar, publicar e gerenciar seus eventos, atividades, inscrições, presença, certificados e indicadores.

Cada conta possui um perfil exclusivo, `PARTICIPANT` ou `ORGANIZER`. A conta organizadora representa a instituição e mantém um único `organizationId`. A mesma conta não alterna entre participação e organização.

Internamente, usuário, instituição, vínculo e `ownerUserId` sustentam a autorização. Papéis administrativos do banco não impõem ao produto um fluxo de formação de equipes. Convites, admissão e delegação entre organizadores não fazem parte do escopo confirmado.

Selecionar outra área não converte o perfil. Após conferir a senha, Identity recusa a área incompatível com `403 account_area_mismatch`, sem emitir token. Participantes podem se inscrever em eventos de instituições diferentes; a restrição institucional pertence à administração.

Evidência: `services/identity/src/users/service.js`:80; `services/events/app/domain.py`:45.

5. Requisitos funcionais

ID	Descrição	Atendimento atual
RF01	Cadastro, autenticação e perfis exclusivos.	Identity, sessão, revogação e área de acesso.
RF02	Gerenciamento dos próprios eventos.	Events: rascunho, publicação, cancelamento e atualização pública.
RF03	Atividades e programação.	Palestrantes, capacidade, preços, lotes e espera.
RF04	Consulta e busca.	API e catálogo; busca/paginação do catálogo no cliente.
RF05	Inscrições em atividades.	Registrations, idempotência e titularidade.
RF06	Reservas, cancelamento e espera.	FIFO, espera configurável e expiração autônoma.
RF07	Pagamento simulado.	Payments e provedor separado; reconciliação.
RF08	Notificações.	Outbox, RabbitMQ, mailbox e suporte SMTP.
RF09	Check-in e presença.	Attendance, registro único e credencial QR.
RF10	Certificados por atividade.	Emissão, PDF, verificação e modelos.
RF11	Relatórios e indicadores.	Ocupação, presença, transações simuladas e CSV.

Também há cadastro institucional transacional, recuperação/troca de senha, sessão de uma hora, confirmação de saída, revalidação da identidade e interface responsiva.

O atendimento identifica implementação substantiva. O alcance da comprovação é apresentado na Seção 18. Pagamentos são simulados. SMTP está implementado, mas recebimento externo não é atribuído a uma nova execução nesta atualização.

6. Requisitos não funcionais

Requisito	Solução e alcance
Modularidade	Serviços separados, dependências e Dockerfiles próprios.
Interoperabilidade	REST/JSON entre Node.js/Express e Python/FastAPI.
Consistência de vagas	Transações, locks, restrições SQL e revisão do inventário.
Duplicidade	Chaves idempotentes, unicidade e recibos persistidos.
Resiliência	Outbox, reconciliação, retry confirmado e filas de falhas.
Observabilidade	Logs, request ID, correlation ID, saúde e rastreamento.
Segurança	Scrypt, JWT RS256, autorização atual e revogação.
Persistência	Nove bancos lógicos e volumes Docker.
Reprodutibilidade	Compose, scripts, migrações e QA separado.
Usabilidade	Interface responsiva, feedback, diálogos e prévia gerenciada.

A separação favorece evolução e escala seletivas, mas não comprova alta disponibilidade. O ensaio de pico de 20 vezes citado na documentação atual não foi medido com base e recursos definidos. Comunicação e Reporting possuem consistência eventual.

7. Histórico de decisões e evolução

O projeto evoluiu do fluxo principal para presença, certificados, indicadores e controles operacionais. O quadro organiza decisões observadas, sem atribuir cronologia experimental não documentada.

Área	Decisão atual
Tecnologias	React/TypeScript, Node.js/Express e Python/FastAPI.
Infraestrutura	Nginx, PostgreSQL, RabbitMQ e Compose.
Identidade	Scrypt, JWT RS256, sessão de uma hora e revogação.
Produto institucional	A própria instituição organiza seus eventos; perfis exclusivos.
Eventos	Programação/preços fixos após publicar; informações públicas atualizáveis.
Inscrições	Capacidade no banco, espera opcional, FIFO e expiração independente.
Pagamento	Provedor simulado separado e decisão persistente recuperável.
Comunicação	Outbox, deduplicação, mailbox e SMTP configurável.
Presença e certificados	Check-in e emissão por atividade concluída.
Indicadores	Projeções e pagamentos por APIs; falha financeira explícita.
Validação/operação	QA separado, preflight, checksum, backup e promoção local.

As referências disponíveis estão em `docs/access-rules.md`, `docs/api/business-rules.md`, `docs/architecture/business-capabilities.md`, `docs/requirements-matrix.md` e nos documentos de implantação e validação.

A redação anterior citava cinco ADRs em `docs/adr/`; esses arquivos não foram encontrados no checkout inspecionado. O relatório não afirma sua existência. Formalizar decisões permanece uma melhoria documental.

8. Tecnologias utilizadas

Frontend

Tecnologia	Finalidade
React 19 e TypeScript 5.9	Componentes e contratos tipados.
Vite 7 e React Router 7	Desenvolvimento, build e navegação.
Context API e CSS	Sessão e responsividade.
Inter / fontes locais	Tipografia e marca.
qrcode e @zxing/browser	Credenciais e leitura QR.
Playwright / Chromium	Testes de interface e navegação.

Backend

Tecnologia	Aplicação
Node.js 24 / Express 5	Gateway, Identity, Payments e provedor.
Python 3.13 / FastAPI	Events, Registrations, Notifications, Attendance, Certificates e Reporting.
Pydantic e Uvicorn	Contratos e execução HTTP Python.
`pg` e `psycopg`	SQL parametrizado; sem ORM identificado.
`jose` / scrypt	JWT e hash de senha.
`express-rate-limit`	Limitação de tentativas.
Pika	RabbitMQ.
Nodemailer / smtplib	Suporte SMTP.
ReportLab / Pillow	PDF e imagens dos certificados.

Infraestrutura

Tecnologia	Finalidade
Docker / Compose	Aplicação e QA com projetos separados.
Nginx	Proxy e distribuição do frontend.
PostgreSQL 17	Nove bancos lógicos na instância local.
RabbitMQ 4	Mensageria e filas duráveis.
Git	Versionamento.
Tunnel, se configurado	Possibilidade de exposição, sem publicação ativa comprovada.

Evidência: `frontend/package.json`; manifests e Dockerfiles; `compose.yaml`.

9. Estrutura de diretórios

O resumo abaixo apresenta os diretórios e pontos de entrada atuais:

```
eventhub/
+-- frontend/
|   +-- public/figma/
|   +-- src/
|   |   +-- auth/
|   |   +-- components/{auth,events,form,profile}/
|   |   +-- pages/{events,organization,participant,registrations}/
|   |   +-- services/
|   |   +-- types/
|   |   +-- App.tsx
|   |   +-- main.tsx
|   +-- tests/access-flows.test.mjs
|   +-- Dockerfile
|   +-- vite.config.ts
|   +-- package.json
+-- gateway/src/{config,middleware,routes,security}/
+-- services/
|   +-- identity/src/
|   +-- events/app/
|   +-- registrations/app/
```

```
|   +-- payments/src/
|   +-- payment-provider/src/
|   +-- notifications/{app,test}/
|   +-- attendance/{app,test}/
|   +-- certificates/{app,test}/
|   +-- reporting/{app,test}/
+-- infra/{db,nginx}/
+-- scripts/
+-- tests/{unit,integration}/
+-- docs/{api,architecture,deployment,validation}/
+-- output/{pdf,visual-audit}/
+-- tmp/
+-- compose.yaml
+-- .env.example
+-- .gitattributes
+-- .gitignore
```

Cada serviço tem manifesto e Dockerfile próprios. `tmp/` contém registros, ambientes e artefatos locais; não representa apenas código autoral nem garante entrega em clone limpo.

O roteiro de interface está em `frontend/tests/README.md`. Não se mantém a afirmação de existência de `playwright.config.ts`, `README.md`, `docs/PLAN.md` ou fontes antigas ausentes deste checkout. Isso não prova inexistência de materiais no ambiente separado de testes.

10. Arquitetura do sistema

O caminho configurado é:

$$\text{Navegador} \rightarrow \text{Nginx} \rightarrow \text{Frontend / Gateway} \rightarrow \text{Serviços.}$$

O frontend consome APIs sob `/api`. O Gateway verifica autenticação e encaminha os contratos; cada domínio mantém suas permissões e regras.

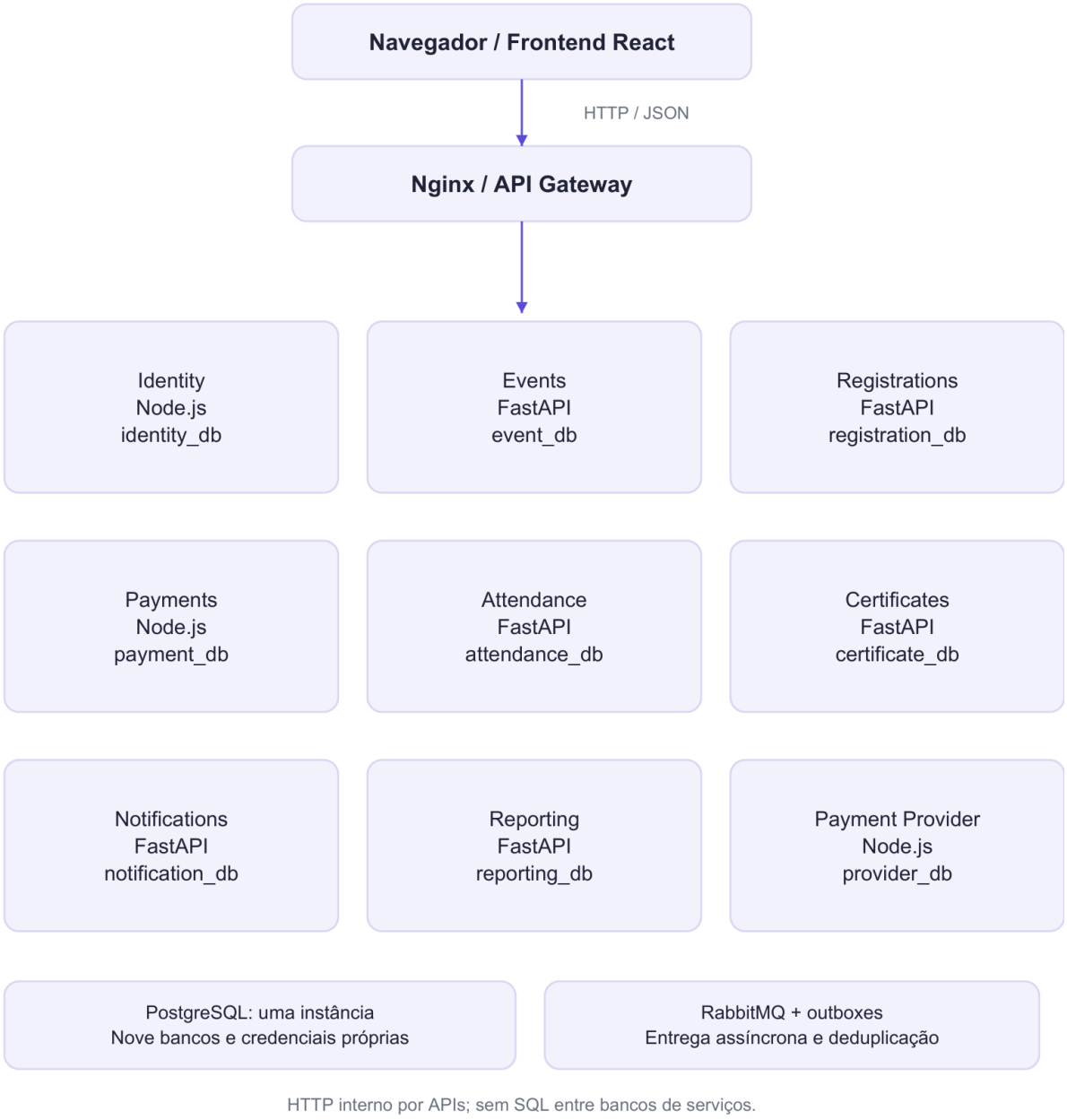


Figura 1: Arquitetura atual: oito domínios, provedor simulado e nove bancos

Chamadas internas utilizam HTTP na rede Docker. Payments consulta o simulador e aplica resultados em Registrations. Attendance, Certificates e Reporting consultam APIs proprietárias. Não há SQL entre bancos de serviços.

Os nove bancos lógicos compartilham uma instância PostgreSQL com usuários próprios. Inscrição e outbox compartilham transação local; provedor e e-mail ficam fora dela. Notifications e Reporting não bloqueiam a confirmação e operam com consistência eventual.

Um túnel pode ser configurado antes do Nginx, mas não é apresentado como publicação ativa verificada.

Organização do frontend

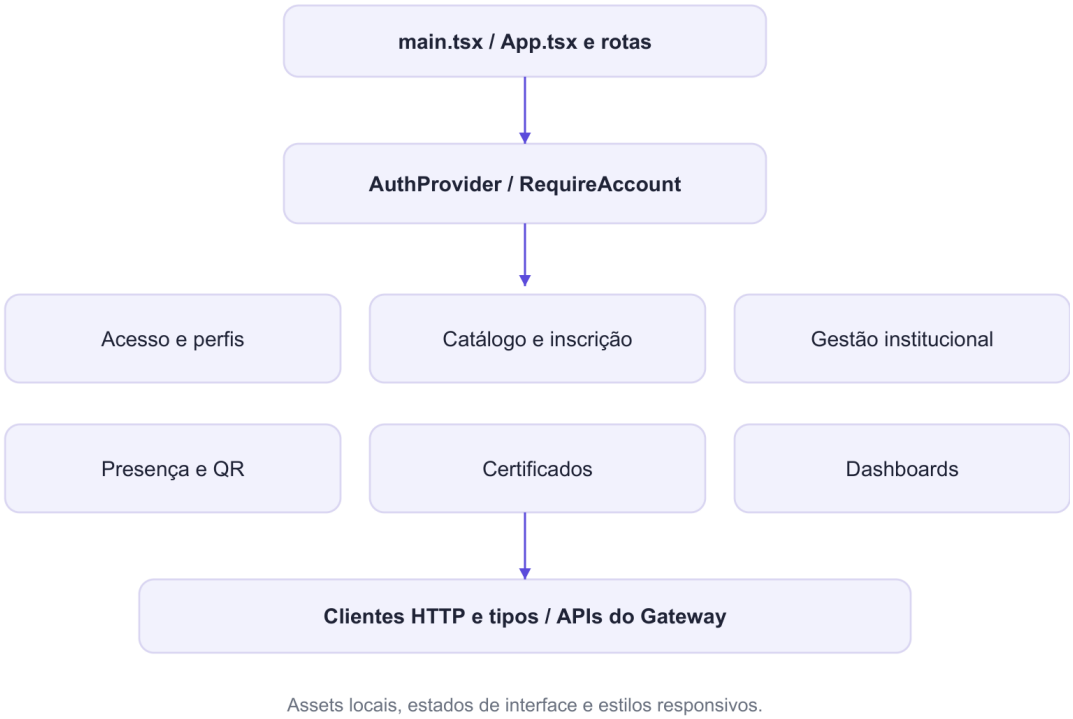


Figura 2: Módulos atuais do frontend e clientes HTTP

Evidência: `gateway/src/routes/business.js`; `docs/architecture/business-capabilities.md`.

11. Descrição detalhada dos módulos

11.1 Frontend

A SPA organiza páginas, componentes, clientes HTTP e tipos. `App` define rotas; `AuthProvider` gerencia e revalida a sessão; `RequireAccount` protege a área; `AppLayout` apresenta navegação e saída.

Há acesso/cadastro, perfis, catálogo, detalhes, editor, checkout, inscrições, gestão institucional, presença, certificados, dashboards e diagnóstico. Os componentes de QR, programação e certificados chamam APIs reais.

11.2 API Gateway

Não possui banco. Encaminha identidade, eventos, disponibilidade, inscrições, pagamentos, presença, certificados, relatórios e downloads. Verifica JWT e identidade atual e propaga correlação. Callbacks e consultas internas não são expostos.

11.3 Identity Service

Mantém contas, credenciais, instituição, vínculo técnico, recuperação, JWT, revogação e confirmação de alterações de e-mail. Cadastro institucional cria conta e instituição atomicamente. Seleção de área e campos arbitrários não alteram perfil/instituição.

11.4 Events Service

Mantém eventos, atividades, palestrantes e preços no JSONB. Valida instituição e responsável. Rascunho aceita edição completa; publicação congela programação, capacidade e preço. Informações públicas podem mudar por contrato específico com avisos aos inscritos.

11.5 Registrations Service

Mantém inventário, inscrições, ações, decisões/recibos e outbox. Reserva, manutenção, cancelamento e pagamento coordenam estados em transação. `expiry_worker` expira sem Events/RabbitMQ; outro worker promove espera, concilia cancelamento e publica.

11.6 Payments Service e provedor simulado

Payments persiste decisão, prazo, correlação e resultado efetivo. O provedor simulado é externo ao domínio Payments, mas pertence ao Docker do projeto e possui banco próprio. Falha temporária mantém reconciliação pendente; não ocorre cobrança real.

11.7 Notifications Service

Consome eventos de inscrição, pagamento e avisos. Valida envelope, resolve destinatário por APIs, registra tentativas, deduplica entregas concluídas e trata retry/DLQ. Mailbox é padrão; SMTP é opção implementada, sem atomicidade com o banco.

11.8 Attendance Service

Mantém presença única por inscrição. Valida autorização institucional, evento publicado, inscrição confirmada e janela da atividade. Resolve credencial QR; confirmar presença é uma operação separada.

11.9 Certificates Service

Valida atividade encerrada e presença correspondente antes de emitir. Persiste snapshot, hash e código público; gera PDF com QR e verifica validade atual. Possui modelos por evento e automação específica de certificados de palestrantes.

11.10 Reporting Service

Mantém projeções e IDs de mensagens processadas. Reconstrói por APIs e retorna inscrições, ocupação, presença, transações simuladas e CSV. Falha financeira é explícita, sem conversão em zero.

12. Funcionamento das funcionalidades

Cadastro e autenticação

Participante informa nome, e-mail e senha; CPF/telefone são opcionais. Instituição informa dados institucionais e obtém perfil `ORGANIZER` com instituição fixa. Login incompatível é recusado sem token ou conversão de perfil.

Uma resposta válida tem a estrutura:

```
{
  "accessToken": "TOKEN",
  "tokenType": "Bearer",
  "expiresIn": 3600,
  "user": {
    "role": "PARTICIPANT",
    "organizationId": null
  }
}
```

Na conta institucional, o perfil é `ORGANIZER` e `organizationId` contém sua instituição.

Sessão, perfil e senha

A sessão tem prazo absoluto de uma hora, sem renovação por reload. O cache é revalidado no servidor antes da área protegida. Saída e expiração limpam a sessão; eventos de armazenamento sincronizam abas. Logout solicita revogação; falha dessa solicitação é informada na interface.

Há edição de nome/CPF/telefone e dados institucionais. Troca autenticada exige senha atual e encerra sessão local; a versão de credenciais invalida tokens antigos. O backend possui troca de e-mail com confirmação, mas a interface atual apresenta e-mail como somente leitura.

Recuperação

Token aleatório de uso único, hash no banco, prazo de 30 minutos e resposta genérica. Mensagens usam mailbox ou SMTP configurado com outbox durável; arquivos da caixa local não possuem consulta pública.

Eventos e inscrição

A instituição cria no contexto fixo, define programação/palestrantes, configura preços, capacidade e espera, salva e publica. Catálogo e detalhes exigem participante autenticado na interface, embora a API de leitura seja pública. A instituição utiliza prévia gerenciada.

Gratuita com vaga confirma; paga gera reserva. Lotação só entra em espera quando habilitada. Checkout envia chave idempotente, recupera estados incertos e identifica o pagamento como simulado.

Presença e certificados

A instituição registra check-in manual ou resolve QR e confirma presença. O participante solicita certificado da atividade encerrada com presença válida. Modelos e certificados de palestrantes são administrados pela instituição. Dashboards e exportações utilizam APIs autorizadas.

Páginas e rotas

Página	Rota
Apresentação / acesso	`/` e `/acesso`
Participante	`/participante/login` e `/participante/cadastro`
Instituição	`/organizacao/login` e `/organizacao/cadastro`
Recuperar / redefinir	`/recuperar-senha` e `/redefinir-senha`
Catálogo / detalhes	`/eventos` e `/eventos/:id`
Gestão / criação	`/organizacao/eventos` e `/organizacao/eventos/novo`
Editor / prévia / operação	`/organizacao/eventos/:id`; sufixos `/visualizar` e `/gestao`
Dashboard	`/organizacao/painel` e `/organizacao/eventos/:id/painel`
Perfis	`/perfil`; `/organizacao/perfil` e `/organizacao/perfil/:id`
Dados institucionais	`/organizacoes`
Checkout de evento	`/eventos/:eventId/inscricao`
Reserva por atividade	`/atividades/:eventId/:activityId/inscricao`
Pagamento / detalhe	`/pagamento/:id` e `/inscricoes/:id`
Inscrições / diagnóstico	`/minhas-inscricoes` e `/diagnostico`

Evidência: `frontend/src/App.tsx`; `frontend/src/pages/registrations/CheckoutPage.tsx`.

13. Regras de negócio

Regra	Aplicação
RN01	Ocupação limitada pela capacidade com proteção concorrente no banco.
RN02	Pagamento confirma apenas reserva válida; gratuita dispensa pagamento.
RN03	Recusa e expiração liberam reserva, sem reativação tardia.
RN04	Cancelamento atualiza ocupação e pode promover FIFO.
RN05	Falha de comunicação não desfaz confirmação.
RN06	Certificado depende de atividade encerrada, inscrição confirmada e presença correspondente.
RN07	Participante acessa seus dados; instituição administra seus eventos.

Controle concorrente

A transação bloqueia inventário com `SELECT FOR UPDATE`. Índices impedem inscrição ativa duplicada e uso indevido de chave. Triggers e revisão persistente reforçam capacidade/transições nos níveis de isolamento suportados.

`CONFIRMED` e `PENDING_PAYMENT` ocupam vaga persistida; `WAITLISTED`, `CANCELLED` e `EXPIRED` não. Reserva pendente continua na proteção do banco até persistir a expiração; mera leitura do relógio não contorna o controle.

Estados e políticas

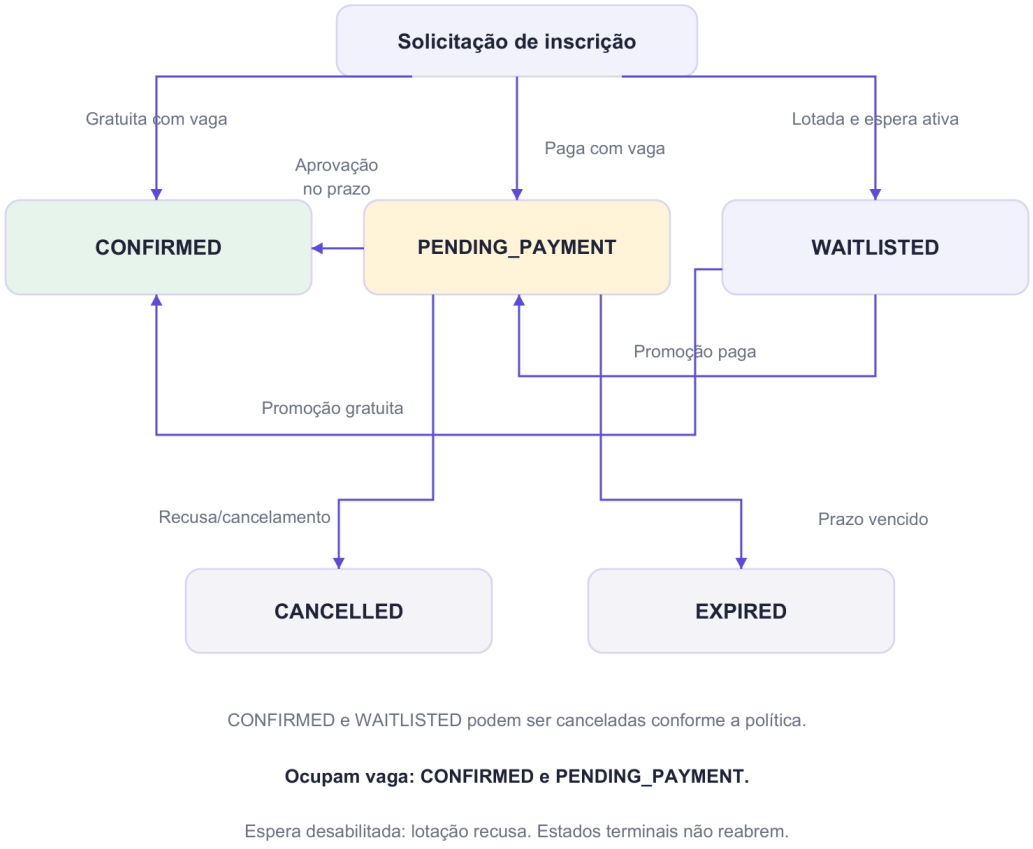


Figura 3: Estados de inscrição e políticas de espera e pagamento

Espera é FIFO por `created_at,id`. Promoção gratuita confirma; paga inicia novo prazo. Espera desabilitada recusa lotação. TTL padrão de 900 segundos é configurável; QA documentado utiliza 120.

Confirmada pode ser cancelada antes do início da atividade. Expiração tem thread própria, dependente apenas de PostgreSQL; lentidão de Events/RabbitMQ não ocupa esse ciclo.

Estados dos pagamentos

Estado	Significado
`PENDING`	Aguardando aplicação/reconciliação.
`APPROVED`	Resultado efetivo confirmou inscrição.
`DECLINED`	Simulação recusada.
`EXPIRED`	Prazo ou estado terminal impede confirmar.

Decisão do provedor e estado efetivo são separados. Aprovação após cancelamento ou expiração não reabre inscrição.

Presença e certificado

Check-in exige evento publicado, inscrição confirmada, autorização e instante do início da atividade até 24 horas após o fim. QR assinado vincula inscrição, pessoa e atividade; resolver QR não grava presença.

Certificado exige término da atividade e presença íntegra. Não se atribuem percentual de frequência, controle de saída ou avaliação não implementados. A certificação por atividade é correta para o produto.

Cancelamento do evento

Events registra e comunica por outbox. Registrations concilia em operações e worker, cancelando inscrições ativas. Há consistência eventual, sem transação SQL distribuída.

14. Modelagem de dados

Propriedade

Banco	Proprietário
`identity_db`	Identity
`event_db`	Events
`registration_db`	Registrations
`payment_db`	Payments
`provider_db`	Payment Provider
`notification_db`	Notifications
`attendance_db`	Attendance
`certificate_db`	Certificates
`reporting_db`	Reporting

Uma instância física hospeda os nove bancos. Referências entre serviços são lógicas e validadas por APIs/eventos; não há chaves estrangeiras distribuídas.

Identity

Tabela	Dados principais
`users`	Nome, e-mail único, hash, perfil, CPF/telefone, preferências, versão de credenciais e datas.
`organizations`	CNPJ, razão social, nome, natureza e contatos.
`organization_members`	Vínculo técnico consistente com perfil; instituição única por usuário organizador.
`password_resets`	Hash do token, usuário e prazo de uso único.
`revoked_sessions`	Hash revogado, usuário e expiração.
`mail_outbox`	Mensagem, estado, tentativas e próxima tentativa.
`email_changes`	Novo endereço, hash, versão e prazo.
`schema_migrations`	Migração, checksum e aplicação.

Events

`events` mantém ID, instituição, `owner_user_id`, estado, `payload` JSONB e criação. Programação, palestrantes e preços são estruturados no payload, sem tabela independente de atividades.

`event_schema_migrations` controla migrações e `event_notification_outbox` persiste avisos.

Registrations

Tabela	Responsabilidade
`inventories`	Atividade, evento, capacidade e revisão.
`registrations`	Usuário, chave, estado, valor, prazo e snapshot.
`outbox`	Envelope e publicação confirmada.
`registration_actions`	Ações administrativas.
`payment_decisions`	Primeira decisão por inscrição.
`payment_notification_receipts`	Recibo e deduplicação do resultado.
`event_announcement_receipts`	Deduplicação de avisos.
`schema_migrations`	Integridade e isolamento.

Payments e provedor

`payments` usa inscrição como chave e mantém usuário, valor, estado, decisão, modo do provedor, IDs, prazo e correlação. `payment_schema_migrations` controla o esquema.

`provider_transactions` pertence ao simulador: referência única, valor, BRL, resultado solicitado, estado e prazo. Nenhuma tabela representa liquidação financeira real.

Demais domínios

Serviço	Estruturas
Notifications	`deliveries`: mensagem única, envelope, estado, tentativas, erro e atualização.
Attendance	`attendance`: inscrição única, usuário, evento, atividade, responsável, instante e snapshot.
Certificates	`certificates`, `certificate_templates` e `speaker_certificates`.
Reporting	`registration_projection`, `event_projection` e `processed_messages`.

Relacionamentos locais

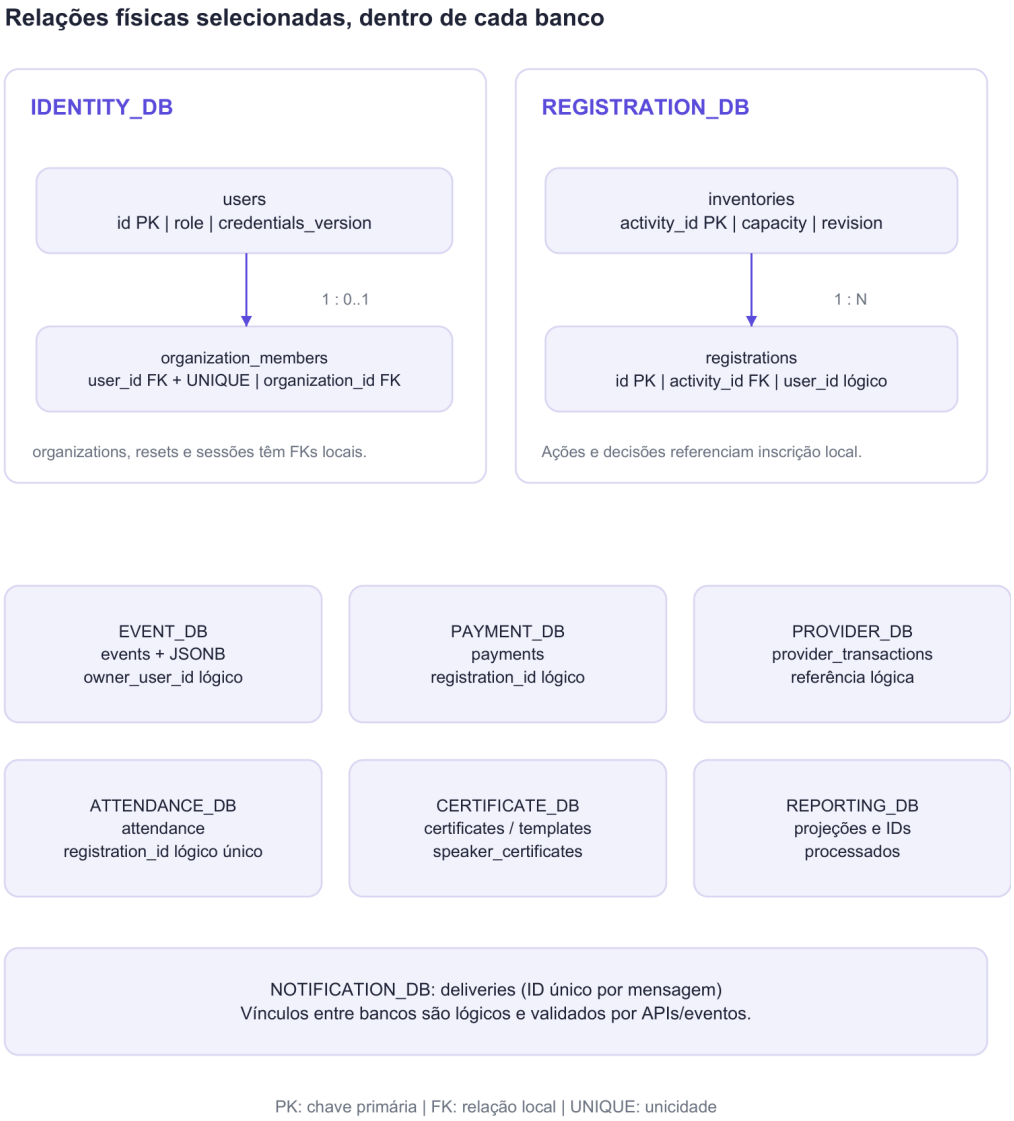


Figura 4: Relações físicas selecionadas e limites dos nove bancos

O diagrama resume relações locais; não substitui todas as colunas/constraints. Os vínculos entre domínios não são FKs distribuídas.

Tipos e contratos

Frontend: `User`, `Session`, `Organization`, `Activity`, `PriceOption`, `Speaker`, `EventData`, `Registration`, `Payment` e `PlatformStatus`, em `frontend/src/types/` e `frontend/src/services/`.

Evidência: `services/identity/src/database/migrations/` e módulos/migrações dos demais domínios.

15. Fluxos de execução

Fluxo principal

1. Participante autentica na área correta e recebe JWT com prazo.
2. Consulta evento, atividade e opção de preço.
3. Frontend solicita inscrição com chave idempotente.
4. Registrations consulta Events e decide sob transação.
5. Gratuita confirma; paga reserva; lotação entra em espera apenas quando habilitada.
6. Pagamento passa por Payments e provedor simulado.
7. Resultado válido atualiza inscrição e outbox.
8. RabbitMQ entrega mensagens para comunicação e projeções.
9. A instituição registra presença na janela permitida; após o término da atividade, o participante pode solicitar o certificado elegível.

Sequência de inscrição paga



Figura 5: Reserva, provedor simulado, reconciliação e comunicação

1. Reserva paga retorna `PENDING_PAYMENT` e prazo.
2. Frontend envia `POST /payments/:id` com resultado simulado.
3. Payments persiste transação pendente, decisão e correlação.
4. Chama `POST /v1/payments` do simulador com credencial e referência idempotente.
5. Provedor retorna aprovação, recusa ou expiração.
6. Payments valida referência, valor, moeda e marcador de simulação.

7. Callback interno em Registrations valida estado/prazo e grava decisão, inscrição e outbox.
8. Payments persiste resultado efetivo; falha temporária mantém reconciliação.
9. Comunicação é posterior e não bloqueia confirmação.

Check-in e certificado

1. Instituição lê ou informa a credencial da atividade.
2. Attendance resolve QR e apresenta dados autorizados.
3. Confirmação separada registra presença única.
4. Participante solicita certificado da atividade concluída.
5. Certificates confere estado, presença e integridade e gera/retorna emissão idempotente.

Falhas

Situação	Comportamento
Área incompatível	403 sem token; não converte perfil.
Evento fora do contexto	Operação negada pelo domínio.
Lotação	Espera se habilitada; caso contrário, recusa.
Recusa/vencimento	Reserva cancelada/expirada e vaga liberada.
Aprovação tardia	Não reabre estado terminal.
Provedor/callback indisponível	Reconciliação pendente.
Broker/consumidor indisponível	Outbox persistida; confirmação preservada.
Mensagem duplicada	ID e estado deduplicam o processamento concluído.
Fonte financeira indisponível	Relatório indica falha, sem falso zero.

16. Integrações externas

Exposição pela internet

Nginx é publicado em loopback na porta 8080. Cloudflare Tunnel ou equivalente pode encaminhar acesso externo para uma demonstração. Não foi comprovada URL ativa ou implantação remota no material examinado.

Figma

Referência visual de acesso, perfis, catálogo, gestão e inscrição. Assets de runtime são locais. Identificadores de nós não comprovam fidelidade visual integral sem baseline de comparação.

Pagamentos

Payment Provider é separado do domínio Payments, mas integra o Docker do projeto. Não é provedor financeiro contratado, não cobra e não expõe suas rotas pelo Gateway.

Notificações e recuperação

`MAIL_MODE=mailbox` é padrão; `MAIL_MODE=smtp` ativa suporte implementado em Identity/Notifications. Remetente, host, credenciais e URL dependem da configuração.

Código SMTP não comprova recebimento externo. Aceitação do e-mail seguida de falha antes de registrar envio pode permitir repetição; não se promete exatamente uma vez.

17. Configuração e execução

Pré-requisitos

- Docker Desktop em modo Linux e Docker Compose.
- Node.js 24 para scripts/desenvolvimento.
- Python compatível fora dos contêineres; imagens Python usam 3.13.
- Chromium para os testes Playwright atuais.

Preparação de ambiente novo

Os comandos descrevem a operação; não foram executados nesta edição:

```
if (-not (Test-Path -LiteralPath '.env')) {
  Copy-Item -LiteralPath '.env.example' -Destination '.env'
}
node scripts/init-backend-config.mjs
node scripts/init-identity-keys.mjs
node scripts/init-internal-key.mjs
```

Configuração inclui nove bancos, RabbitMQ, JWT, chave interna e do simulador, e-mail, URL e TTL. Scripts preservam valores existentes. Segredos reais não devem ser versionados.

Inicialização e portas

```
docker compose up --build -d --wait --wait-timeout 600
docker compose ps
# Aplicacao: http://127.0.0.1:8080
# Prontidao: http://127.0.0.1:8080/api/status
```

Componente	Porta interna
Nginx / frontend	80
Gateway, Identity, Payments e provedor	3000
Serviços Python	8000
PostgreSQL	5432
RabbitMQ	5672

Somente HTTP do Nginx é publicado no host, em `127.0.0.1:8080`. Prontidão não substitui validação de negócio.

QA separado

```
node scripts/init-rules-qa.mjs
docker compose -p eventhub-rules-qa --env-file tmp/rules-qa/.env up -d --build --wait --wait-
    timeout 600
# QA: http://127.0.0.1:18080
```

QA possui configuração, chaves, dados e volumes próprios. Ensaios de falha devem permanecer nesse destino.

Desenvolvimento e encerramento

```
npm --prefix frontend ci
npm --prefix frontend run dev
npm --prefix frontend run build
docker compose logs --tail 100
docker compose down
```

Vite encaminha `/api` à porta 8080. Encerramento normal preserva volumes; não remover volumes para contornar migração.

Migrações e backup

Identity, Events, Registrations e Payments possuem migrações com checksum e controle transacional. Preflights recusam legado ambíguo. Outros domínios inicializam suas tabelas.

Volume existente pode exigir novos bancos; alterar `.env` não muda automaticamente senhas persistidas. Runbook documenta diagnóstico, backup/restauração e promoção. Consistência entre serviços exige coordenar gravações, outboxes e filas.

Evidência: `docs/deployment/business-rules-runbook.md`; `scripts/init-backend-config.mjs`; `scripts/init-rules-qa.mjs`.

18. Testes e evidências de validação

Testes integrados são executados em QA separado. O projeto Compose `eventhub-rules-qa`, na porta 18080, tem configuração, credenciais, chaves, bancos e volumes próprios. A aplicação usa `eventhub`, na porta 8080.

QA documentado utiliza TTL de 120 segundos; produto possui padrão configurável de 900. Esta edição organiza registros existentes, sem executar nova regressão completa.

Níveis de validação

Nível	Alcance
Unitário/domínio	Regras com dependências simuladas; não prova concorrência PostgreSQL real.
Interface interceptada	Telas e contratos com respostas HTTP simuladas.
Integração QA	HTTP, banco e serviços reais, fixtures e falhas controladas.
Navegação integrada	Chromium contra QA e capturas desktop/celular.
Operação local	Imagens, migrações e prontidão; não substitui regras de negócio.

Suítes e comandos atuais

```
node --test tests/unit/identity-access.test.mjs
npm --prefix frontend run test:access
node --test tests/integration/capacity-isolation.mjs
node --test tests/integration/access-migrations.mjs
node --test tests/integration/business-rules.mjs
node --test tests/integration/payment-recovery.mjs
node tests/integration/frontend-live.mjs
node --test tests/integration/login-area.mjs
node tests/integration/trace-evidence.mjs
```

Os integrados pressupõem preparação e seleção de QA. Python possui `tests/unit/test_event_access.py` e `test_rules.py` em Attendance, Certificates, Reporting e Notifications.

A suíte de interface tem 14 casos com APIs interceptadas; não cobre automaticamente toda a lista antiga de navegador. Ausência local de material não prova inexistência no ambiente separado.

Resultados históricos de 10/11/2026

Verificação	Aprovados	Observação
Identity	6	Revisão anterior ao ajuste de login.
Acesso Events	6	Responsabilidade e autorização isoladas.
Domínios adicionais	27	Attendance 5; Certificates 10; Reporting 10; Notifications 2.
Interface interceptada	14	Acesso, checkout e dashboard.
Isolamento de capacidade	5	Documentado; quatro cenários e agrupador.
Migrações de acesso	4	Log presente; três cenários e agrupador.
Negócio HTTP	8	Log presente; sete cenários e agrupador.
Recuperação de pagamento	3	Log presente; dois cenários e agrupador.

O total histórico de 73 inclui agrupadores; não são 73 cenários independentes nem nova regressão da revisão atual. Logs: `tmp/rules-qa/integration-results.log`, `tmp/rules-qa/access-migrations-results.log` e `tmp/rules-qa/payment-recovery-results.log`.

Navegação QA real registrou sete verificações, 16 capturas em 1440/390 pixels e `errors=[]`, referentes à revisão observada.

Validações posteriores e limites

O ajuste de login em 11/11 registra oito testes Identity, 14 de interface e quatro integrados QA aprovados. `output/visual-audit/2026-11-11/login-area/verification.json` informa origem 18080 e quatro capturas. Os demais integrados não foram repetidos para esse ajuste; não somar indiscriminadamente aos 73.

Na verificação técnica isolada de 11/11/2026, passaram 41 testes: Identity 8, Events 6, Attendance 5, Certificates 10, Reporting 10 e Notifications 2. TypeScript terminou com saída zero, sem emissão. Esses resultados são distintos do QA integrado e não foram repetidos nesta edição documental.

Expiração usa fixture que torna prazo vencido; não mede todo o TTL natural. Outbox durante falha não prova SMTP recebido ou drenagem integral. Restauração de nove bancos em clones demonstra recuperação técnica, sem provar consistência distribuída completa.

Evidência: `docs/validation/implementation-summary.md`:60–110; `docs/validation/domain-rules-results.md`; `docs/deployment/business-rules-runbook.md`.

19. Estado atual do desenvolvimento

Componente	Estado atual
Frontend	Acesso por perfil, catálogo, checkout, inscrições, gestão institucional e dashboards.
Identity	Cadastro, perfil, instituição fixa, recuperação, sessão e revogação.
Events	Eventos, programação, palestrantes, preços e informações públicas.
Registrations	Capacidade, idempotência, reserva, espera, expiração e outbox.
Payments/Provider	Simulação persistente e reconciliação, sem cobrança real.
Notifications	Consumo, deduplicação, retry, mailbox e SMTP configurável.
Attendance	Check-in autorizado, manual ou QR, e presença única.
Certificates	Atividade, emissão, PDF, verificação, modelos e palestrantes.
Reporting	Projeções, ocupação, presença, transações simuladas e CSV.
Infraestrutura/QA	14 contêineres, nove bancos e testes em ambiente separado.

Perfis exclusivos, instituição fixa e certificado após atividade são regras corretas. Equipes não são exigidas pelo produto.

A consulta local da elaboração do relatório de estado atual registrou 14 contêineres saudáveis e HTTP 200/ready em `/api/status`. Promoção de imagens e checksums pertencem a registros históricos, não a monitoramento contínuo.

O resultado é um protótipo acadêmico integrado, sem cobrança real, implantação remota comprovada ou alta disponibilidade demonstrada.

20. Limitações e melhorias futuras

Attendance, Certificates e Reporting já estão implementados. As limitações concentram-se em maturidade operacional e evidências específicas.

Limitação	Evolução
Pagamento simulado	Integração financeira real com cobrança e reembolso definidos.
SMTP externo não demonstrado	Comprovar envio e recebimento controlados.
Sessão em localStorage	Avaliar cookie HttpOnly e proteções correspondentes.
Uma instância PostgreSQL	Separar recursos conforme disponibilidade/carga medidas.
Programação fixa após publicar	Contrato coordenado se surgir requisito de alteração.
Workers e consistência eventual	Métricas, alertas e recuperação de backlog/filas.
Implantação remota não comprovada	Definir destino e validação posterior.
Carga 20 vezes não medida	Definir base, recursos, duração e critérios.
Algumas listagens no cliente	Paginação/filtros no servidor conforme volume.

- Automatizar CI e regressão associadas a uma revisão identificável.
- Consolidar evidências de QA com ambiente, data e alcance.
- Ampliar backup para linhas, invariantes, papéis e coordenação das filas.
- Validar recuperação completa de mensagens, além da persistência durante falha.
- Documentar rotação de segredos e monitorar dependências.
- Alinhar terminologia antiga de equipes e certificados.
- Apresentar números de avaliações da interface apenas com fonte ou identificação demonstrativa.

Revogação, SMTP, rastreabilidade e backup têm implementação. Não se atribuem percentual de frequência, controle de saída, assinatura ICP-Brasil ou cobrança real sem requisito e implementação.

21. Comparação entre planejamento e estado atual

Planejamento anterior	Estado atual	Interpretação
Plataforma de eventos	Catálogo, programação e gestão institucional.	Objetivo preservado.
Cinco serviços operacionais	Oito domínios e provedor simulado.	Escopo ampliado.
Duas stacks	Node/Express e Python/FastAPI.	Decisão preservada.
Cinco bancos	Nove bancos lógicos.	Novos domínios persistem dados.
Conta e vínculos de organização	Perfis exclusivos e conta institucional.	Mudança intencional.
Gestão de equipe	Instituição gerencia seus eventos.	Equipe fora do escopo.
Certificado após evento	Após atividade e presença válida.	Regra atual correta.
Espera na lotação	Espera configurável.	Política ampliada.
Três serviços modelados	Presença, certificados e relatórios operacionais.	Evolução incorporada.
Comunicação apenas simulada	Mailbox e suporte SMTP.	Entrega externa exige validação.
JWT sem revogação central	Revogação e versão de credenciais.	Segurança evoluída.
Backup/rastreio futuros	Ferramentas e registros locais.	Avanço operacional.
Estratégia de validação	QA separado, testes em camadas.	Ambiente/revisão acompanham resultado.
Tunnel previsto	Possibilidade, sem URL comprovada.	Não afirmar publicação ativa.

Mudanças intencionais foram incorporadas sem recomendar reversão de regras corretas. Resultados históricos mantêm o alcance de suas revisões, sem equivaler a uma execução nova.

22. Encerramento e próximos passos recomendados

O EventHub implementa identidade, eventos, inscrições, pagamento simulado, notificações, presença, certificados e indicadores. Preserva propriedade de dados e tratamento explícito de capacidade e falhas distribuídas.

A instituição é a conta organizadora de seus eventos; participação usa perfil próprio. Certificados seguem a conclusão de cada atividade com presença válida. Os testes integrados pertencem ao QA separado.

1. Consolidar revisão de entrega, relatório, contratos, diagramas e instruções.
2. Preservar evidências com data, revisão, ambiente e limites.
3. Repetir regressão pertinente quando novas alterações justificarem.
4. Comprovar SMTP, recuperação integral de filas e backup coordenado quando necessários.
5. Definir parâmetros de carga antes de atribuir desempenho.
6. Validar publicação remota apenas no destino configurado.
7. Ampliar automação, monitoramento e segurança conforme uso real.

A entrega supera o recorte de cinco serviços e três apenas modelados. Deve mostrar separadamente implementação, validação em QA e limites operacionais, sem inventar resultados.

Referências técnicas da atualização

Caminhos relativos ao repositório inspecionado. Código, teste isolado, evidência histórica e consulta de prontidão têm alcances diferentes. As decisões confirmadas pelo responsável em 11/11/2026 prevalecem sobre redações antigas.

Área	Fontes
Arquitetura	`compose.yaml`; `docs/architecture/business-capabilities.md`; `infra/db/01-init.sh`.
Acesso	`services/identity/src/users/service.js`; `services/identity/src/security/`; `services/events/app/domain.py`.
Regras de inscrição	`services/registrations/app/domain.py` e `services/registrations/app/migrations/`.
Pagamento	`services/payments/src/payments.js`; `services/payment-provider/src/app.js`.
Presença/certificados	`services/attendance/app/domain.py`; `services/certificates/app/`.
Comunicação/relatórios	`services/notifications/app/`; `services/reporting/app/domain.py`.
Interface	`frontend/src/App.tsx`; páginas, componentes e serviços em `frontend/src/`.
QA e operação	`docs/deployment/business-rules-runbook.md`; `docs/validation/implementation-summary.md`; `docs/validation/domain-rules-results.md`.
Evidências	`tmp/rules-qa/`; `output/visual-audit/`; caminhos indicados na seção de testes.

Esta atualização preserva as 22 seções e o estilo do texto fornecido. É uma edição documental: não modifica implementação nem executa novas suítes ou migrações.